DANH SÁCH MODULE RTL

Platform: FPGA Basys3 (Artix-7) | Interface: SPI | Format: INT8 | PE Array: 4×4

Tổng: 12 module | 3 nhóm dễ nhất để bắt đầu: argmax.v → post_proc.v → spi_slave_rx.v

TỔNG QUAN CÁC MODULE

#	Module	Nhóm	Độ khó	Ghi chú
1	spi_slave_rx.v	Giao tiếp I/O	⭐ Dễ	
2	spi_slave_tx.v	Giao tiếp I/O	⭐⭐ Trung bình	
3	input_buffer.v	Bộ nhớ / Buffer	⭐⭐ Trung bình	
4	weight_rom.v	Bộ nhớ / Buffer	⭐⭐⭐ Khó	
5	acc_buffer.v	Bộ nhớ / Buffer	⭐⭐⭐ Khó	
6	pe.v	Tính toán	⭐⭐ Trung bình	
7	pe_array.v	Tính toán	⭐⭐⭐ Khó	
8	pool.v	Tính toán	⭐⭐ Trung bình	
9	post_proc.v	Hậu xử lý	⭐ Dễ	
10	argmax.v	Hậu xử lý	⭐ Dễ	
11	controller.v	Điều phối	⭐⭐⭐⭐ Rất khó	Viết sau cùng
12	top.v	Tích hợp	⭐⭐⭐⭐ Rất khó	Chỉ nối dây

THỨ TỰ VIẾT RTL KHUYẾN NGHỊ

argmax.v → post_proc.v → spi_slave_rx.v → spi_slave_tx.v → pe.v → pool.v
→ input_buffer.v → weight_rom.v → acc_buffer.v → pe_array.v → controller.v → top.v

Nguyên tắc: module sau phụ thuộc module trước. Verify xong mỗi module mới đi tiếp.

NHÓM 1 — GIAO TIẾP I/O

1. spi_slave_rx.v

Nhận 784 bytes ảnh từ ESP32 qua chuẩn SPI Mode 0. Xử lý clock domain crossing giữa SCLK (ESP32) và system clock FPGA.

Bảng tín hiệu:

Tín hiệu	Độ rộng	Chiều	Mô tả
clk	1	IN	System clock FPGA (50 MHz)
rst	1	IN	Reset đồng bộ, active high
SCLK	1	IN	SPI clock từ ESP32 (bất đồng bộ với clk)
MOSI	1	IN	Data nối tiếp từ ESP32
CS_N	1	IN	Chip select, active low
data_out	8	OUT	1 byte vừa nhận xong
data_valid	1	OUT	Pulse 1 cycle khi data_out hợp lệ

frame_done	1	OUT	Pulse khi nhận đủ 784 bytes (1 ảnh hoàn chỉnh)

Lưu ý thiết kế:

⚠ *SCLK không đồng bộ với clk — cần 2-FF synchronizer bên trong*

⚠ *Sample MOSI tại cạnh lên của SCLK (SPI Mode 0: CPOL=0, CPHA=0)*

⚠ *CS_N low = bắt đầu frame, CS_N high = kết thúc frame*

2. spi_slave_tx.v

Gửi 1 byte kết quả (class 0–9) về ESP32 qua SPI MISO sau khi inference xong.

Bảng tín hiệu:

Tín hiệu	Độ rộng	Chiều	Mô tả
clk	1	IN	System clock FPGA
rst	1	IN	Reset đồng bộ, active high
SCLK	1	IN	SPI clock từ ESP32
CS_N	1	IN	Chip select, active low
data_in	8	IN	Byte kết quả từ argmax (class 0–9)
send_en	1	IN	Pulse báo "có data cần gửi" — từ controller
MISO	1	OUT	Data nối tiếp gửi về ESP32
tx_done	1	OUT	Pulse khi gửi xong 8 bit

Lưu ý thiết kế:

⚠ *Shift out MSB trước, đổi bit tại cạnh xuống SCLK*

⚠ *data_in phải ổn định trước khi send_en được pulse*

NHÓM 2 — BỘ NHỚ / BUFFER

3. input_buffer.v

Lưu 784 bytes ảnh nhận từ spi_slave_rx vào BRAM nội bộ. Cho phép controller đọc từng pixel theo địa chỉ.

Bảng tín hiệu:

Tín hiệu	Độ rộng	Chiều	Mô tả
clk	1	IN	System clock
rst	1	IN	Reset đồng bộ
wr_data	8	IN	Pixel từ spi_slave_rx.data_out
wr_en	1	IN	Nối với spi_slave_rx.data_valid
rd_addr	10	IN	Địa chỉ đọc pixel (0–783) từ controller
rd_data	8	OUT	Pixel INT8 tại địa chỉ rd_addr
buf_full	1	OUT	Pulse khi đã nhận đủ 784 bytes — báo controller bắt đầu inference

Lưu ý thiết kế:

⚠ *Địa chỉ ghi tự tăng mỗi khi wr_en = 1, reset khi frame_done*

⚠ *buf_full kết nối với controller.buf_full_in để kích hoạt FSM*

4. weight_rom.v

Lưu toàn bộ weight INT8 của LeNet-5 (~61 KB) trong BRAM. Khởi tạo từ file .coe sinh bởi Python. Read-only.

Bảng tín hiệu:

Tín hiệu	Độ rộng	Chiều	Mô tả
clk	1	IN	System clock
layer_sel	3	IN	Chọn layer: 0=Conv1, 1=Conv2, 2=FC1, 3=FC2, 4=FC3
rd_addr	16	IN	Địa chỉ weight trong layer đang chọn
weight_out	8	OUT	Weight INT8 (signed) tại địa chỉ yêu cầu — latency 1 cycle

Lưu ý thiết kế:

⚠ *Mỗi layer map sang 1 vùng địa chỉ tuyệt đối riêng trong BRAM*

⚠ *layer_sel + rd_addr → controller sinh ra, không phải PE array*

⚠ *Không có cổng ghi — dùng BRAM initialization (.coe file)*

5. acc_buffer.v

Lưu partial sum trung gian (32-bit) và output feature map giữa các layer. Đóng 2 vai trò: accumulate trong khi tính, lưu output làm input layer tiếp theo.

Bảng tín hiệu:

Tín hiệu	Độ rộng	Chiều	Mô tả
clk	1	IN	System clock
rst	1	IN	Reset đồng bộ
wr_data	32	IN	Partial sum từ pe_array hoặc kết quả sau post_proc (8-bit zero-extended)
wr_addr	12	IN	Địa chỉ ghi — từ controller
wr_en	1	IN	Write enable
rd_addr	12	IN	Địa chỉ đọc — từ controller
rd_en	1	IN	Read enable
rd_data	32	OUT	Partial sum hoặc activation tại địa chỉ đọc — latency 1 cycle

Lưu ý thiết kế:

⚠ *32-bit để giữ đủ precision cho partial sum chưa requantize*

⚠ *Sau post_proc + pool, kết quả 8-bit ghi ngược vào đây làm input layer tiếp*

⚠ *Cần đủ dung lượng cho feature map lớn nhất: 28×28×6 = 4,704 entries*

NHÓM 3 — TÍNH TOÁN

6. pe.v

Processing Element cơ bản. Thực hiện 1 phép MAC: psum_out = psum_in + (weight × activation). Có pipeline register 1 stage để tránh timing violation.

Bảng tín hiệu:

Tín hiệu	Độ rộng	Chiều	Mô tả
clk	1	IN	System clock
rst	1	IN	Reset đồng bộ
weight_in	8	IN	Weight INT8 signed — giữ cố định trong Weight Stationary
act_in	8	IN	Activation INT8 signed — chạy qua PE mỗi cycle
psum_in	32	IN	Partial sum vào từ PE phía trên (0 nếu đầu cột)
en	1	IN	Enable MAC — khi 0 giữ nguyên psum
load_weight	1	IN	Pulse nạp weight_in vào register nội bộ
psum_out	32	OUT	psum_in + (weight × activation) — registered output

Lưu ý thiết kế:

⚠ *weight_in × act_in = tích 16-bit, sign-extend lên 32-bit trước khi cộng*

⚠ *Pipeline 1 stage: kết quả ra sau 1 cycle từ lúc en = 1*

⚠ *Module nhỏ nhất, viết và verify đầu tiên*

7. pe_array.v

Kết nối 4×4 = 16 PE theo lưới 2D, dataflow Weight Stationary. Thực hiện 16 phép MAC song song mỗi cycle.

Bảng tín hiệu:

Tín hiệu	Độ rộng	Chiều	Mô tả
clk	1	IN	System clock
rst	1	IN	Reset đồng bộ
weight_in	4×8	IN	4 weights nạp vào 4 cột (1 hàng weight mỗi lần load)
act_in	4×8	IN	4 activations nạp vào 4 cột, chảy xuống theo hàng
en	1	IN	Enable toàn bộ array
load_weight	1	IN	Pulse nạp weight mới vào tất cả PE
psum_out	4×32	OUT	4 partial sum output từ hàng cuối cùng
array_valid	1	OUT	Pulse khi psum_out hợp lệ (sau pipeline delay)

Lưu ý thiết kế:

⚠ *Activation chảy từ trên xuống theo cột (systolic)*

⚠ *Partial sum tích lũy dọc theo cột từ PE hàng 0 xuống hàng 3*

⚠ *Module quan trọng nhất về timing — cần kiểm tra timing report sau synthesis*

8. pool.v

Average Pooling 2×2: lấy trung bình 4 pixel liền kề → 1 pixel output. Chạy độc lập với PE array, không dùng chung tài nguyên.

Bảng tín hiệu:

Tín hiệu	Độ rộng	Chiều	Mô tả
clk	1	IN	System clock
rst	1	IN	Reset đồng bộ
data_in	8	IN	Pixel INT8 từ post_proc output, gửi tuần tự
wr_en	1	IN	Ghi pixel vào line buffer nội bộ
row_size	8	IN	Chiều rộng feature map: 28 (sau Conv1) hoặc 10 (sau Conv2)
start	1	IN	Pulse bắt đầu pooling — từ controller
data_out	8	OUT	Pixel đã pooling (trung bình 4 pixel, INT8)
data_valid	1	OUT	Pulse khi data_out hợp lệ
pool_done	1	OUT	Pulse khi pooling xong toàn bộ feature map

Lưu ý thiết kế:

⚠ *Cần line buffer 1 hàng bên trong để giữ hàng trên khi xử lý hàng dưới*

⚠ *Chỉ dùng cho Conv1 và Conv2 — FC layer không có pooling*

⚠ *Average: cộng 4 pixel rồi shift right 2 bit (chia 4)*

NHÓM 4 — HẬU XỬ LÝ

9. post_proc.v

Xử lý sau PE array: ReLU → requantize (shift right) → clamp về INT8. Pipeline 3 stage.

Bảng tín hiệu:

Tín hiệu	Độ rộng	Chiều	Mô tả
clk	1	IN	System clock
rst	1	IN	Reset đồng bộ
acc_in	32	IN	Partial sum 32-bit từ acc_buffer
scale	8	IN	Scale factor INT8 của layer hiện tại — hardcode từ Python
shift	4	IN	Số bit shift right khi requantize — từ controller
valid_in	1	IN	Pulse khi acc_in hợp lệ
result_out	8	OUT	Kết quả INT8 sau ReLU + requantize
valid_out	1	OUT	Pulse khi result_out hợp lệ (sau pipeline delay)

Lưu ý thiết kế:

⚠ *Stage 1: ReLU — nếu acc_in[31]=1 (âm) → clamp về 0*

⚠ *Stage 2: nhân scale rồi shift right → giảm về 8-bit range*

⚠ *Stage 3: clamp [-128, 127] → cast INT8*

⚠ *scale và shift khác nhau theo từng layer — controller cấp giá trị đúng*

10. argmax.v

So sánh 10 score đầu ra của FC3, tìm index lớn nhất → class dự đoán 0–9. Module đơn giản nhất, viết đầu tiên.

Bảng tín hiệu:

Tín hiệu	Độ rộng	Chiều	Mô tả
clk	1	IN	System clock
rst	1	IN	Reset đồng bộ
score_in	8	IN	1 score INT8 mỗi cycle, gửi tuần tự 10 cycle
score_valid	1	IN	Pulse mỗi khi score_in hợp lệ
start	1	IN	Pulse reset max_val và index, bắt đầu so sánh mới
class_out	4	OUT	Index class thắng (0–9)
valid_out	1	OUT	Pulse khi đã so sánh xong cả 10 scores

Lưu ý thiết kế:

⚠ *Giữ max_val và max_idx trong register, cập nhật mỗi khi score_valid = 1*

⚠ *valid_out pulse sau score_valid lần thứ 10*

⚠ *Dễ nhất trong 12 module — chỉ cần comparator và 2 register*

NHÓM 5 — ĐIỀU PHỐI

11. controller.v

FSM trung tâm điều phối toàn bộ inference pipeline. Sinh địa chỉ memory và control signal cho từng module theo từng layer. Module phức tạp nhất — viết sau cùng.

Bảng tín hiệu:

Tín hiệu	Độ rộng	Chiều	Mô tả
clk	1	IN	System clock
rst	1	IN	Reset đồng bộ
buf_full	1	IN	Từ input_buffer — ảnh đã nhận đủ
array_valid	1	IN	Từ pe_array — partial sum hợp lệ
pool_done	1	IN	Từ pool — pooling xong
valid_out_pp	1	IN	Từ post_proc — output hợp lệ
valid_out_am	1	IN	Từ argmax — class xác định xong
rd_addr_input	10	OUT	Địa chỉ đọc pixel từ input_buffer

rd_addr_weight	16	OUT	Địa chỉ đọc weight từ weight_rom
layer_sel	3	OUT	Chọn layer cho weight_rom
wr_addr_acc	12	OUT	Địa chỉ ghi vào acc_buffer
rd_addr_acc	12	OUT	Địa chỉ đọc từ acc_buffer
wr_en_acc	1	OUT	Write enable cho acc_buffer
rd_en_acc	1	OUT	Read enable cho acc_buffer
pe_en	1	OUT	Enable pe_array
load_weight	1	OUT	Pulse nạp weight vào PE
pool_start	1	OUT	Bắt đầu pooling
row_size	8	OUT	Chiều rộng feature map cho pool.v
scale	8	OUT	Scale factor cho post_proc
shift	4	OUT	Shift amount cho post_proc
valid_in_pp	1	OUT	Báo post_proc có data cần xử lý
argmax_start	1	OUT	Reset và bắt đầu argmax
send_en	1	OUT	Báo spi_slave_tx gửi kết quả

Lưu ý thiết kế:

⚠ *FSM states: IDLE → LOAD → CONV1 → POOL1 → CONV2 → POOL2 → FC1 → FC2 → FC3 → ARGMAX → SEND → IDLE*

⚠ *Tách thành 2 tầng: top FSM (layer sequence) + sub FSM (address generation trong mỗi layer)*

⚠ *Module khó nhất — chỉ bắt đầu sau khi tất cả module khác đã verify xong*

NHÓM 6 — TÍCH HỢP

12. top.v

Module tích hợp toàn bộ hệ thống. Không có logic tính toán — chỉ instantiate và nối dây 11 module còn lại.

Bảng tín hiệu:

Tín hiệu	Độ rộng	Chiều	Mô tả
clk	1	IN	System clock từ oscillator Basys3 (100 MHz, chia xuống 50 MHz bên trong)
rst	1	IN	Nút reset trên Basys3
SCLK	1	IN	SPI clock từ ESP32
MOSI	1	IN	SPI data từ ESP32
CS_N	1	IN	SPI chip select từ ESP32
MISO	1	OUT	SPI data về ESP32
DONE_GPIO	1	OUT	GPIO báo ESP32 inference xong, có thể đọc kết quả

Lưu ý thiết kế:

⚠ *DONE_GPIO = argmax.valid_out — kết nối thẳng 1 chân GPIO ra pin FPGA*

⚠ *Thêm clock divider (100 MHz → 50 MHz) bên trong top.v*

LUỒNG DỮ LIỆU TỔNG THỂ

Data path (data thực sự chạy qua):

```
ESP32
  → spi_slave_rx  (nhận 784 bytes ảnh)
  → input_buffer  (lưu tạm)
  → pe_array  ←  weight_rom  (tính MAC song song)
  → acc_buffer  (tích lũy partial sum)
  → post_proc  (ReLU + requantize về INT8)
  → pool.v  (Average Pooling 2×2 — chỉ sau Conv1, Conv2)
  → acc_buffer  (lưu output layer, làm input layer tiếp
theo)
  [lặp lại cho Conv2, FC1, FC2, FC3]
  → argmax  (tìm class max trong 10 scores)
  → spi_slave_tx  → ESP32  (trả về 1 byte class 0–9)
```

Control path (controller sinh signal điều phối, KHÔNG mang data):

```
controller  →  rd_addr, wr_addr, layer_sel, en, start  →
tất cả module
controller  ←  buf_full, array_valid, pool_done, valid_out
←  tất cả module
```

Lưu ý quan trọng:

⚠ *Chỉ Conv1 đọc từ input_buffer. Conv2 trở đi đọc từ acc_buffer.*

⚠ *acc_buffer đóng 2 vai trò: lưu partial sum đang tính VÀ lưu output feature map giữa các layer.*

⚠ *FC layer không có pooling. Controller tắt pool_start cho FC1, FC2, FC3.*

⚠ *Sau Pool2, feature map 5×5×16 = 400 phần tử được flatten thành vector 1D — controller đọc tuần tự địa chỉ, không cần module riêng.*